Logo

Le logo du logiciel représente une combinaison entre un **engrenage industriel** et un **aigle stylisé**, ce qui reflète parfaitement l'objectif du projet.

L'engrenage symbolise le monde industriel et les systèmes SCADA, en mettant en avant les machines, les processus et l'automatisation. Il représente également la précision et le fonctionnement continu des systèmes industriels.

Au centre, on distingue un **aigle**, qui symbolise la vigilance, la surveillance et la puissance d'analyse. L'aigle est souvent associé à la vision et à la capacité de détecter des menaces à distance, ce qui correspond parfaitement à la fonction de détection d'anomalies et de sécurité du logiciel.

La forme globale intègre également les initiales du système, donnant une identité unique au logiciel.

Méthodologie du projet (modèle en cascade adapté)

Dans ce projet, j'ai suivi une démarche structurée inspirée du **modèle en cascade**, avec des étapes bien définies allant de l'analyse jusqu'à la démonstration finale.

🔹 Phase 1 — Analyse

Au début, j'ai commencé par comprendre le problème.

✔ Ce que j'ai fait :

- définir le sujet du projet
- identifier le problème : manque de sécurité dans les systèmes SCADA
- fixer les objectifs :
  - supervision
  - détection d'anomalies
  - audit
- définir la valeur ajoutée :
  - ajouter une couche cybersécurité à un SCADA

👉 Cette phase m'a permis de bien cadrer le projet avant de commencer le développement.

🔹 Phase 2 — Étude

Ensuite, j'ai étudié l'existant.

✔ Ce que j'ai étudié :

- fonctionnement des systèmes SCADA
- protocole Modbus TCP
- menaces OT (attaques industrielles, anomalies)
- solutions existantes (SCADA classiques)

👉 Cette étape m'a aidé à comprendre les limites des systèmes actuels et à orienter mon projet vers la détection.

🔹 Phase 3 — Spécification

Dans cette phase, j'ai défini précisément ce que le système doit faire.

✔ Fonctionnalités :

- login utilisateur
- dashboard
- lecture Modbus
- détection
- alertes
- audit

✔ Rôles :

- utilisateur (visualisation)
- administrateur (analyse complète)

✔ Flux :

- PLC → backend → analyse → dashboard

✔ Contraintes :

- temps réel
- simplicité
- modularité

👉 Cette phase permet d'éviter les erreurs pendant le développement.

🔹 Phase 4 — Conception

Après, j'ai conçu le système avant de coder.

✔ Ce que j'ai conçu :

- architecture globale du projet
- structure base de données (SQLite)
- diagrammes UML :
  - cas d'utilisation
  - classes
  - séquence
- interfaces (dashboard)

👉 Cette étape permet d'avoir une vision claire du système.

🔹 Phase 5 — Développement

Ensuite, j'ai développé le projet en plusieurs parties.

✔ Frontend :

- interface avec Vue.js
- dashboard
- affichage des données

✔ Backend :

- Node.js + Express
- API
- gestion utilisateurs

✔ Communication :

- lecture Modbus
- connexion PLC simulé

✔ Détection :

- logique d'anomalies

- analyse des données

👉 Le développement a été fait progressivement module par module.

🔹 Phase 6 — Test

Après le développement, j'ai testé le système.

✔ **Tests réalisés :**

- communication Modbus
- affichage dashboard
- détection anomalies
- génération alertes
- audit

👉 L'objectif est de vérifier que tout fonctionne correctement.

🔹 Phase 7 — Livraison

Enfin, j'ai préparé la phase finale.

✔ **Livrables :**

- rapport du projet
- présentation (PPT)
- démonstration du système

👉 Cette étape permet de présenter le travail final au jury.

Flux de fonctionnement

📊 Fonctionnement complet

1. Le PLC simulé génère des données
2. Le backend lit les données via Modbus
3. Les données sont stockées dans SQLite

4. Le moteur d'analyse traite les données
5. Le backend reçoit le résultat
6. Le frontend affiche :
    - valeurs
    - alertes
    - risque

Technologies utilisées

🖥️ 1. Electron

Electron est un framework qui permet de développer des **applications desktop** en utilisant des technologies web comme HTML, CSS et JavaScript.

👉 Il combine :

- un moteur d'affichage (Chromium)
- un environnement serveur (Node.js)

Utilisation dans le projet

Electron permet de transformer l'application en **logiciel de supervision SCADA**, utilisable comme une application installée sur un poste industriel.

🌐 2. Vue.js

Vue.js est un framework JavaScript utilisé pour créer des **interfaces utilisateur interactives**.

Caractéristiques

- structure basée sur des composants
- mise à jour dynamique
- simple à utiliser

Utilisation dans le projet

Vue.js est utilisé pour :

- créer le dashboard

- afficher les variables industrielles
- gérer l'interface utilisateur

🧠 3. Node.js

Node.js est un environnement qui permet d'exécuter du JavaScript côté serveur.

Caractéristiques

- rapide
- asynchrone
- adapté aux applications temps réel

Utilisation dans le projet

Node.js est utilisé pour :

- gérer le backend
- traiter les requêtes
- communiquer avec le frontend
- lire les données industrielles

🔧 4. Express.js

Express est un framework qui facilite le développement d'applications web avec Node.js.

Caractéristiques

- gestion simple des routes
- organisation du code
- middleware

Utilisation dans le projet

Express permet :

- de créer des API REST
- de structurer le backend

🗄️ 5. SQLite

SQLite est une base de données légère intégrée directement dans l'application.

Caractéristiques

- ne nécessite pas de serveur
- rapide
- facile à utiliser

Utilisation dans le projet

SQLite est utilisée pour stocker :

- les utilisateurs
- les données SCADA
- les alertes
- les résultats d'analyse

🏭 6. Modbus TCP

Modbus TCP est un protocole de communication utilisé dans les systèmes industriels.

Caractéristiques

- basé sur TCP/IP
- simple
- largement utilisé

Utilisation dans le projet

Modbus TCP permet :

- de communiquer avec le PLC
- de lire les variables industrielles

⚙️ 7. Simulateur PLC (Modbus Slave)

Un simulateur PLC est un outil qui reproduit le comportement d'un automate industriel.

Caractéristiques

- génère des données
- simule un environnement industriel
- configurable

Utilisation dans le projet

Il permet :

- de tester le système sans matériel réel
- de simuler des situations normales et anormales

🧠 8. Rust

Rust est un langage de programmation moderne orienté performance et sécurité.

Caractéristiques

- très rapide
- sécurisé (gestion mémoire)
- fiable

Utilisation dans le projet

Rust est utilisé pour :

- analyser les données
- détecter les anomalies
- calculer le risque

🧠 9. Python

Python est un langage simple et très utilisé en analyse de données.

Caractéristiques

- facile à apprendre
- nombreuses bibliothèques
- rapide pour prototypage

Utilisation dans le projet

Python peut être utilisé pour :

- tester des règles de détection
- analyser les données

🐳 10. Docker

Docker est une plateforme qui permet de créer et exécuter des applications dans des conteneurs.

Caractéristiques

- isolation des services
- portabilité
- déploiement facile

Utilisation dans le projet

Docker permet :

- d'isoler les composants
- de simplifier le lancement du projet

☸️ 11. Kubernetes

Kubernetes est un système d'orchestration de conteneurs.

Caractéristiques

- gestion multi-services
- automatisation
- scalabilité

Utilisation dans le projet

Kubernetes permet :

- d'organiser les services
- de simuler une architecture industrielle distribuée

🧪 12. REST Api

Postman est un outil utilisé pour tester les API.

Utilisation dans le projet

- tester les routes backend
- vérifier les réponses

💻 13. Visual Studio Code

VS Code est un éditeur de code.

Utilisation

- écrire le code
- gérer les fichiers
- debug

🔄 14. Git

Git est un système de gestion de version.

Utilisation

- sauvegarder le code
- suivre les modifications
- collaborer